

Guía de Git y GitHub

Material de apoyo · MCDI500 — Programación para la Ciencia de Datos

Entorno de desarrollo con Python y JupyterLab

Versión para macOS



Magíster en Ciencia de Datos e Inteligencia Artificial

Facultad de Ingeniería · Universidad Andrés Bello

2026

Contenidos

Contenidos	2
Cómo usar esta guía.....	5
Ruta de lectura por fase y semana.....	5
Por qué esto se evalúa: mapa guía ↔ rúbrica	7
PARTE I. Conceptos previos.....	7
1.1 El problema que Git resuelve	8
1.2 ¿Qué es Git?	8
1.3 ¿Qué es GitHub?	8
PARTE II. Instalación y preparación del entorno	9
2.1 Instalar las herramientas de línea de comandos de Xcode.....	9
2.2 Instalar Git.....	9
2.3 Instalar Python 3	9
2.4 Configurar Git por primera vez	10
PARTE III. Crear el proyecto y su entorno	10
3.1 Crear la carpeta del proyecto	11
3.2 Crear y activar un entorno virtual	11
3.3 Instalar la base de trabajo	11
3.4 Registrar el kernel del proyecto.....	11
3.5 Extensiones e integración con Git.....	12
3.6 Inicializar Git en el proyecto	12
3.7 Estructura de carpetas del proyecto del curso	12
3.8 Crear los archivos iniciales.....	12
PARTE IV. Los tres estados de Git y el primer flujo.....	13
4.1 Los tres estados.....	13
4.2 El primer flujo completo.....	13
PARTE V. El archivo .gitignore.....	14
PARTE VI. Notebooks y Git: higiene para ciencia de datos	15
6.1 Solución 1: limpiar salidas con nbstripout	15
6.2 Solución 2: emparejar con Jupyter	16
PARTE VII. Reproducibilidad del entorno	16
7.1 Reproducibilidad no es solo el requirements.txt.....	17
PARTE VIII. GitHub: cuenta y repositorio del grupo.....	17

8.1 Crear la cuenta.....	17
8.2 Crear el repositorio remoto	18
8.3 Agregar a los integrantes del grupo	18
PARTE IX. Conectar Git local con GitHub.....	18
9.1 Agregar el remoto	18
9.2 Verificar el remoto.....	19
9.3 Autenticarse con GitHub CLI.....	19
9.4 Subir el proyecto por primera vez	19
PARTE X. Flujo de trabajo en equipo.....	19
10.1 El ciclo individual	19
10.2 El ciclo en grupo.....	19
PARTE XI. Ramas.....	20
11.1 ¿Qué es una rama?	20
11.2 Crear y cambiar de rama.....	20
11.3 Trabajar y guardar en la rama	21
PARTE XII. Colaboración con Pull Requests	21
12.1 Fusión local (alternativa simple)	21
PARTE XIII. Conflictos: qué son y cómo resolverlos	22
13.1 Cómo se ve	22
13.2 Cómo se resuelve	22
13.3 Conflictos en notebooks.....	22
PARTE XIV. El README técnico.....	23
14.1 Plantilla	23
PARTE XV. Mensajes de commit.....	24
15.1 La convención del curso.....	24
15.2 Qué distingue un buen mensaje	24
PARTE XVI. Errores comunes y su significado	25
PARTE XVII. Integridad académica en el repositorio	27
PARTE XVIII. Checklist antes de cada entrega	27
PARTE XIX. Secuencia consolidada desde cero	28
19.1 Del inicio a la publicación.....	28
19.2 Incorporarse a un repositorio existente (resto del grupo)	28
19.3 Desarrollo con ramas y Pull Request	29
Glosario.....	29

Autoevaluación	30
Conclusión	30

Cómo usar esta guía

Tiempo de lectura | 90 min en total (no está pensada para leerse de una sola vez: consulte cada parte cuando la necesite, según la ruta de la sección siguiente).

Esta guía acompaña las cuatro fases del proyecto transversal del módulo. No es un manual de referencia para consultar salteado: está ordenada como una secuencia, y cada parte supone que usted completó la anterior.

Trabaje con la guía abierta al lado de la Terminal. Copie los comandos, pero **léalos antes de ejecutarlos**: entender qué hace cada uno es parte de lo que se evalúa. Un repositorio construido a ciegas se nota en la revisión —los commits lo delatan— y no le sirve cuando algo falle a mitad de la Fase 3.

La guía asume que usted parte desde cero, sin Python ni Git instalados.

Nota — ¿usa Windows? Existe una versión equivalente de esta guía para Windows 10 y 11. Los conceptos y los comandos de Git son idénticos; cambian la instalación, la terminal y la activación del entorno virtual. Use la versión que corresponda a su equipo.

Convenciones

Recuadro	Significado
Nota	Información complementaria que aclara el concepto.
Atención	Punto donde se producen la mayoría de los errores. Léalo antes de ejecutar.
Recomendación	Práctica profesional que conviene adoptar desde ya.
Rúbrica	Vínculo explícito con un criterio evaluado del curso.
Comprueba	Verificación de cierre de cada parte: si no puede responderla, vuelva atrás.

Ruta de lectura por fase y semana

El módulo dura tres semanas y usted no necesita la guía completa desde el primer día. Esta tabla indica qué partes corresponden a cada momento del curso y para qué le sirven.

Momento del curso	Partes	Para qué le sirve
Fase 0 — Diagnóstico	I	Comprender qué problema resuelve el control de versiones antes de instalar nada.
Semana 1 • Fase 1 — Formativa 1 (mapa conceptual) y Sumativa 1	II a V, VIII, IX, XIV	Instalar el entorno, crear el repositorio del grupo, definir la estructura de carpetas y el README.
Semana 1 • Fase 2 — segunda parte de la Sumativa 1	VI, VII, X, XV	Higiene de notebooks, reproducibilidad del entorno y flujo de trabajo en equipo.
Semana 2 • Fase 3 —	XI, XII, XIII	Ramas, Pull Requests y resolución de conflictos, cuando el trabajo grupal se

Momento del curso	Partes	Para qué le sirve
Formativa 3 y Sumativa 3		intensifica.
Semana 3 • Fase 4 — Sumativa 4	XIV, XVIII	README definitivo, trazabilidad completa y checklist de entrega final.
En cualquier momento	XVI, XVII, glosario	Errores frecuentes, integridad académica y vocabulario.

Recomendación — antes de la Formativa 1 La Formativa 1 le pide representar en un mapa conceptual el **flujo de trabajo reproducible** del proyecto. Las Partes III, V y VII de esta guía describen exactamente ese flujo: estructura de carpetas, qué se versiona y qué no, y cómo se garantiza que otra persona pueda ejecutar el proyecto. Léalas antes de elaborar el mapa y tendrá la mitad del trabajo hecho.

Si usted ya trabaja con Git y GitHub

El grupo de estudiantes que ingresa al Magíster llega con experiencias muy distintas. Si usted ya usa Git con soltura, no necesita leer la guía completa: verifique que domina estos cinco puntos y salte directamente a las partes indicadas.

¿Sabe hacerlo?	Si no	Si sí
Explicar la diferencia entre <code>git pull</code> y <code>git fetch</code>	Parte X	—
Agregar colaboradores a un repositorio y trabajar los cuatro sobre el mismo	Parte VIII	—
Evitar que los notebooks generen conflictos ilegibles	Parte VI	Esta suele ser la brecha incluso en perfiles con experiencia
Resolver un conflicto de merge sin entrar en pánico	Parte XIII	—
Escribir un README que permita a un tercero ejecutar el proyecto	Parte XIV	—

En todos los casos, revise la **Parte XVIII (checklist de entrega)** antes de cada evaluación: contiene los requisitos específicos de este curso, que no dependen de su experiencia previa con la herramienta.

Dónde pedir ayuda

Esta guía no reemplaza los canales del curso. Si algo no funciona después de intentarlo:

- **Foro de Consultas** — para dudas sobre contenidos y comandos. Antes de preguntar, indique qué comando ejecutó, qué esperaba y qué mensaje exacto recibió: así la respuesta llega en la primera iteración.
- **Foro de Tutoría** — para problemas de acceso, plataforma o soporte técnico.
- **Cápsula 1: Implementa tu entorno reproducible paso a paso** y el **video de Jupyter** de la Semana 1 — cubren en formato audiovisual lo mismo que las Partes II

y III. Si prefiere ver antes que leer, empiece por ahí y use la guía como referencia escrita.

Por qué esto se evalúa: mapa guía ↔ rúbrica

Git y GitHub no son un requisito administrativo del curso. Son la evidencia de cómo trabajó usted. En un proyecto de ciencia de datos, el resultado final no basta: se exige poder demostrar cómo se llegó a él, quién hizo qué y que otra persona pueda reproducirlo. Eso es exactamente lo que registra un repositorio bien llevado.

En las evaluaciones del módulo, varios criterios dependen directamente de lo que cubre esta guía:

Criterio de la rúbrica	Qué mira el revisor	Parte de esta guía
Control de versiones	Historial verificable, commits descriptivos y frecuentes, uso de ramas cuando corresponde	Partes IV, V, XI, XV
Repositorio GitHub	Carpetas por fase, README estructurado, historial completo, commits identificables por cada integrante	Partes VIII, IX, X, XIV
Reproducibilidad técnica	Ambiente virtual, dependencias completas y verificadas, ejecución sin errores en cualquier equipo	Partes III, VII
Documentación del proceso	Decisiones técnicas registradas, flujo de trabajo explicado, cambios justificados con motivo e impacto	Partes XIV, XV
Notebook ejecutado y documentado	Notebook que corre de principio a fin, con trazabilidad clara hacia el repositorio	Partes VI, VII

Rúbrica El criterio de Repositorio GitHub exige **contribuciones individuales identificables por integrante**. Un repositorio donde todos los commits los realizó una sola persona no alcanza el nivel Excelente, aunque el contenido sea impecable. Esto no se puede corregir la noche anterior a la entrega: el historial refleja cuándo trabajó cada quien. Comiencen a registrar commits desde la primera semana.

Atención — equipos mixtos Es habitual que en un grupo de cuatro convivan equipos con macOS y con Windows. El repositorio, los comandos de Git y el `requirements.txt` funcionan igual en ambos; lo que cambia es la ruta de activación del entorno virtual (`.venv/bin/activate` en macOS, `.venv/Scripts/activate` en Windows). Documenten **ambas** en el README para que nadie del equipo quede bloqueado.

PARTE I. Conceptos previos

Tiempo de lectura | 5 min

Al terminar esta parte usted podrá explicar con sus palabras qué problema resuelve Git, y por qué Git y GitHub no son lo mismo.

1.1 El problema que Git resuelve

Antes de saber qué es Git, conviene reconocer el problema. Es probable que ya lo haya vivido:

```
analisis_final.ipynb
analisis_final_v2.ipynb
analisis_final_v2_CORREGIDO.ipynb
analisis_final_ESTESI.ipynb
analisis_final_ESTESI_camila.ipynb
```

Esa carpeta no permite responder tres preguntas básicas: qué cambió entre una versión y otra, por qué se hizo ese cambio, y cuál era el estado del proyecto el día en que los resultados sí funcionaban. En un trabajo grupal el problema se multiplica, porque además nadie sabe qué versión tiene el resto.

1.2 ¿Qué es Git?

Git es un **sistema de control de versiones**: registra los cambios de un proyecto a lo largo del tiempo. Con Git usted sabe qué modificó, cuándo, por qué —si escribió buenos mensajes— y puede recuperar cualquier estado anterior.

En ciencia de datos esto es especialmente valioso. Un experimento que ayer daba un resultado y hoy da otro casi siempre se explica mirando qué cambió en el historial: una transformación distinta, un filtro agregado, una semilla que se movió.

1.3 ¿Qué es GitHub?

GitHub es una plataforma en la nube que aloja repositorios Git. Git trabaja localmente, en su computador; GitHub permite respaldar el proyecto en internet, compartirlo, colaborar y centralizar el historial del equipo.

Nota En simple: **Git** = control de versiones, funciona en su máquina, sin internet. **GitHub** = copia remota, colaboración y revisión. Se puede usar Git sin GitHub. No al revés.

Atención — error conceptual frecuente GitHub no es «una carpeta en la nube» ni un iCloud Drive para código. Si lo utiliza solo para subir el archivo final, el repositorio quedará con uno o dos commits gigantes y sin historia. La rúbrica evalúa el **historial**, no la existencia del archivo: un repositorio usado como almacenamiento corresponde al nivel más bajo del criterio.

Comprueba ¿Podría explicarle a un compañero, sin mirar la guía, por qué renombrar archivos con `_v2` no equivale a versionar?

PARTE II. Instalación y preparación del entorno

Tiempo de lectura | 10 min · Semana 1, Fase 1

Al terminar esta parte tendrá Git y Python funcionando en su equipo, y Git sabrá quién es usted.

En macOS todo el trabajo se hace desde la aplicación **Terminal** (Aplicaciones → Utilidades → Terminal, o Spotlight con + espacio escribiendo «Terminal»).

Nota Es posible que haya visto guías que hablan de **Git Bash**: esa es la terminal que acompaña a Git en Windows. En macOS no existe Git Bash ni instaladores .exe; se usa directamente la Terminal del sistema, con los mismos comandos de Git.

2.1 Instalar las herramientas de línea de comandos de Xcode

Antes que nada, conviene instalar las **Command Line Tools**, que incluyen un compilador y muchas dependencias que las librerías científicas necesitan. En la Terminal:

```
xcode-select --install
```

Acepte la ventana que aparece. Si ya están instaladas, el sistema se lo indicará.

2.2 Instalar Git

Hay dos formas habituales en macOS.

Opción A — Command Line Tools de Xcode. El paso anterior normalmente ya instala Git. Verifíquelo con:

```
git --version
```

Si Git no estuviera instalado, macOS le ofrecerá instalarlo; acepte.

Opción B — Homebrew. Si usted usa Homebrew, el gestor de paquetes de macOS:

```
brew install git
```

Nota Verifique con `git --version`. Debería mostrar algo similar a `git version 2.x.x`. El número exacto no importa: basta con que el comando responda.

2.3 Instalar Python 3

Descargue Python desde el sitio oficial: <https://www.python.org/>.

Atención En macOS el comando `python` a secas no existe, y `command not found` es el primer tropiezo de casi todos. Mientras no active un entorno virtual, use siempre **python3** y **pip3**.

Verifique la instalación con:

```
python3 --version
```

Recomendación — qué versión instalar No instale la versión más reciente solo por ser la más nueva: las librerías científicas tardan meses en dar soporte completo a cada versión de Python. Para este módulo, **Python 3.13 es la opción segura**. Python 3.14 ya cuenta con soporte en el stack principal —scikit-learn 1.8, por ejemplo, admite de 3.11 a 3.14— pero si le aparece un error de compilación al instalar una librería, la causa más probable es esa. Evite versiones anteriores a 3.11.

Atención — Mac con chip Apple (M1 a M4) Para que PyTorch use la GPU (backend mps) necesita macOS 12.3 o superior y un Python compilado de forma **nativa para ARM64**, no la versión emulada en x86 mediante Rosetta. Un Python emulado hace que la GPU se reporte como no disponible aunque el hardware sí la tenga; podrá comprobarlo más adelante con `torch.backends.mps.is_available()`. Para trabajar con pandas y scikit-learn en este módulo no necesita nada de esto.

2.4 Configurar Git por primera vez

Git necesita saber quién realiza los commits. Configure nombre y correo, y deje main como rama principal por defecto:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global init.defaultBranch main
```

Verifique:

```
git config --global --list
```

Atención Use **el mismo correo con el que creará su cuenta de GitHub**. Si no coinciden, sus commits aparecerán en el historial sin vincularse a su perfil, y en la revisión no será posible atribuirle su trabajo. Corregirlo después implica reescribir el historial, que es un procedimiento delicado.

Nota `--global` significa que la configuración aplica a todos los repositorios de su usuario en ese equipo. Definir `init.defaultBranch main` evita tener que renombrar la rama más adelante.

Comprueba Ejecute `git config --global user.email`. ¿Es el correo que usará en GitHub?

PARTE III. Crear el proyecto y su entorno

Tiempo de lectura | 12 min · Semana 1, Fase 1

Al terminar esta parte tendrá la carpeta del proyecto con un entorno virtual propio y JupyterLab funcionando dentro de él.

Atención — el orden importa Primero se crea la carpeta del proyecto, dentro de ella el entorno virtual, y **recién entonces** se instalan los paquetes. Así nunca se toca el Python del sistema —que en macOS responde con el error `externally-managed-environment` si se intenta— y cada proyecto queda aislado con sus propias versiones.

3.1 Crear la carpeta del proyecto

Para el proyecto del curso, usen el nombre acordado por el grupo:

```
mkdir proyecto-grupoX-mcdi500
cd proyecto-grupoX-mcdi500
```

3.2 Crear y activar un entorno virtual

Un entorno por proyecto evita conflictos entre versiones de paquetes:

```
python3 -m venv .venv
source .venv/bin/activate
```

Cuando el entorno está activo verá `(.venv)` al inicio de la línea. **A partir de ese momento sí puede usar python y pip a secas**, porque apuntan al entorno y no al Python del sistema.

Nota Para salir del entorno: `deactivate`. Recuerde **volver a ejecutar `source .venv/bin/activate` cada vez que abra una terminal nueva** en el proyecto. Si instala un paquete y «desaparece», casi siempre es porque el entorno no estaba activo.

3.3 Instalar la base de trabajo

```
python -m pip install --upgrade pip
python -m pip install jupyterlab notebook ipykernel \
    numpy pandas matplotlib seaborn scikit-learn
```

Abra JupyterLab con:

```
python -m jupyter lab
```

3.4 Registrar el kernel del proyecto

Esto permite que cada notebook use el entorno correcto.

La terminal anterior quedó ocupada ejecutando JupyterLab, así que **abra una segunda terminal** (⌘+ N en Terminal), sitúese de nuevo en la carpeta del proyecto con `cd`, active el entorno y ejecute:

```
source .venv/bin/activate
python -m ipykernel install --user --name grupoX_mcdi500 \
    --display-name "Python (grupoX-mcdi500)"
```

Atención Al crear un notebook, elija **el kernel de su proyecto**, no el genérico «Python 3». Es la causa número uno del clásico «a mí me funciona y a ti no»: cada integrante ejecutando contra un intérprete distinto.

3.5 Extensiones e integración con Git

```
python -m pip install jupyterlab-git jupyter nbstripout
```

- **jupyterlab-git** añade un panel de Git dentro de JupyterLab.
- **jupyter** empareja un notebook `.ipynb` con un `.py`, lo que facilita versionar y revisar el código.
- **nbstripout** limpia las salidas de los notebooks antes de cada commit (Parte VI).

3.6 Inicializar Git en el proyecto

```
git init
```

Esto crea un repositorio Git local en la carpeta. Desde este momento Git puede rastrear los cambios.

3.7 Estructura de carpetas del proyecto del curso

El proyecto transversal avanza por fases y el repositorio debe reflejar esa progresión. La rúbrica evalúa que la estructura sea coherente con el mapa conceptual técnico elaborado en la Formativa 1.

```
proyecto-grupoX-mcdi500/  
  F1/  
    data/raw/           # datos originales, sin modificar  
    data/processed/     # datos tras limpieza y transformación  
    notebooks/F1_Definicion.ipynb  
    src/                # funciones reutilizables (.py)  
    docs/               # documentación y referencias  
  F2/  
  F3/  
  F4/  
  requirements.txt  
  .gitignore  
  README.md
```

Rúbrica El criterio de Repositorio GitHub en la Sumativa 1 pide explícitamente **carpetas F1 y F2**, README estructurado e historial completo. No improvisen la estructura: definirla en la Fase 1 y respetarla evita reorganizaciones masivas que ensucian el historial.

Recomendación Nunca modifique los archivos de `data/raw/`. Los datos originales son la referencia contra la cual se verifica todo lo demás; si los sobrescribe, pierde la posibilidad de demostrar qué transformó.

3.8 Crear los archivos iniciales

```
touch README.md
```

Para el notebook, en JupyterLab: desde el *Launcher*, sección **Notebook**, o desde **File → New → Notebook**. Elija el kernel del proyecto, renómbrelo y guárdelo en `F1/notebooks/`.

El `.gitignore` lo crearemos con contenido en la Parte V, **antes del primer commit**.

Comprueba ¿Ve (`.venv`) en su terminal? ¿El notebook que creó muestra «Python (grupoX-mcdi500)» como kernel en la esquina superior derecha?

PARTE IV. Los tres estados de Git y el primer flujo

Tiempo de lectura | 8 min · Semana 1, Fase 1

Al terminar esta parte entenderá por dónde pasa un archivo antes de quedar guardado en el historial, y habrá realizado su primer commit.

4.1 Los tres estados

Git trabaja con tres estados. Entenderlos evita la mayoría de las confusiones posteriores.

Working Directory (directorio de trabajo). Donde usted crea, edita y guarda archivos. Git detecta los cambios pero todavía no los ha registrado.

Staging Area (zona de preparación). Zona intermedia donde se colocan los cambios que se quieren incluir en el próximo commit. Existe para poder elegir qué entra: si modificó tres archivos y solo dos corresponden al cambio que desea registrar, prepara esos dos.

Committed (confirmado en el historial). Git guardó esa versión de forma permanente, con autor, fecha y mensaje.

edita archivo $\longrightarrow$ git add staging $\longrightarrow$ git commit historial

Nota Un **commit** es un registro guardado de cambios: una fotografía del proyecto en un momento concreto. Git deja constancia de qué cambió, cuándo, con qué mensaje y quién lo hizo.

4.2 El primer flujo completo

Revisar el estado. Es el comando que más utilizará:

```
git status
```

Muestra archivos nuevos, modificados y preparados. Ejecútelo antes y después de cada operación mientras esté aprendiendo: es la forma de que Git deje de parecer una caja negra.

Preparar los cambios:

```
git add nombre_archivo    # un archivo específico
git add .                  # todo lo que cambió
```

Confirmar:

```
git commit -m "Crea estructura inicial del proyecto"
```

Ver el historial:

```
git log
git log --oneline
git log --oneline --graph --all    # útil cuando haya ramas
```

Recomendación `git add .` es cómodo pero peligroso si no revisó antes con `git status`. Es la vía habitual por la que se sube por accidente la carpeta `.venv/` completa o un archivo de datos de 500 MB.

Comprueba Ejecute `git log --oneline`. ¿Aparece su commit con su nombre? Verifique el autor con `git log -1 --format='%an <%ae>'`.

PARTE V. El archivo .gitignore

Tiempo de lectura | 5 min · Semana 1, Fase 1

Al terminar esta parte su repositorio ignorará lo que no debe versionarse.

El `.gitignore` indica qué elementos Git no debe rastrear: entornos, caché, datos pesados y archivos sensibles. Créelo con este contenido para un proyecto de ciencia de datos en macOS:

```
# Entorno virtual
.venv/

# Caché de Python
__pycache__/
*.pyc

# Jupyter
.ipynb_checkpoints/

# macOS
.DS_Store
.AppleDouble

# Variables de entorno y secretos
.env

# Datos pesados y modelos
*.ckpt
*.pth
```

Atención Cree y complete el `.gitignore` **antes de su primer `git add ..`** Si no, corre el riesgo de versionar la carpeta `.venv/` (cientos de archivos) o un `.env` con

credenciales. El archivo `.DS_Store` lo genera macOS en cada carpeta que usted abre en el Finder; ignorarlo mantiene el repositorio limpio y evita conflictos absurdos con los compañeros.

Atención — datos Decidan esto como grupo y déjenlo escrito en el README.

Si el dataset es liviano (unos pocos MB), versionarlo facilita que cualquiera reproduzca el análisis y es lo preferible para este curso. Si es pesado, ignórenlo y documenten en el README de dónde se descarga y cómo se coloca en la carpeta, porque el criterio de Reproducibilidad exige que otra persona pueda ejecutar el proyecto completo.

Nota GitHub rechaza archivos de más de 100 MB y advierte sobre los mayores a 50 MB. Si su dataset supera ese tamaño, no intente forzarlo: documente la fuente.

Comprueba Ejecute `git status`. ¿Aparece `.venv/` o `.DS_Store` en la lista? Si aparecen, su `.gitignore` no está funcionando: revise que esté en la raíz del proyecto y bien escrito.

PARTE VI. Notebooks y Git: higiene para ciencia de datos

Tiempo de lectura | 8 min · Semana 1, Fase 2

Al terminar esta parte sus notebooks dejarán de generar diferencias ilegibles y conflictos constantes.

Este es el punto que más problemas causa en los proyectos grupales y que las guías genéricas omiten. Un archivo `.ipynb` guarda, dentro del mismo JSON, **las salidas de cada celda y los contadores de ejecución**. Eso provoca tres problemas:

- **Diffs ilegibles.** Cambiar una línea puede generar cientos de líneas de diferencia, porque también cambian las imágenes codificadas y los números de ejecución.
- **Conflictos de merge constantes** y difíciles de resolver: dos integrantes que solo ejecutaron el mismo notebook, sin editarlo, ya tienen versiones distintas.
- **Filtración de datos.** Una tabla o un gráfico impreso puede dejar información sensible dentro del repositorio público.

6.1 Solución 1: limpiar salidas con nbstripout

Con nbstripout ya instalado, actívalo en el repositorio una sola vez:

```
nbstripout --install
```

Desde ese momento, cada `git add` de un notebook guardará la versión sin salidas. El notebook en pantalla no se altera; solo se limpia lo que va al historial.

Atención — coordinación grupal `nbstripout --install` configura el repositorio **local de cada integrante**. Debe ejecutarlo cada persona del grupo después de clonar, o el beneficio se pierde en cuanto uno haga commit sin él.

Atención — y la evaluación La rúbrica exige un **notebook ejecutado**, con resultados visibles. Con nbstripout activo, el .ipynb del repositorio no tendrá salidas. Resuélvanlo así: mantengan el notebook limpio en el repositorio, y **exporten a PDF o HTML el notebook ejecutado** para adjuntarlo al informe (File → Save and Export Notebook As). Así cumplen ambas cosas: historial limpio y evidencia de ejecución.

6.2 Solución 2: emparejar con Jupyter

Como alternativa o complemento, puede vincular cada notebook a un archivo .py de texto plano, versionar el .py y dejar el .ipynb fuera de Git:

```
jupyter --set-formats ipynb,py:percent F1/notebooks/F1_Definicion.ipynb
```

Esto crea F1_Definicion.py sincronizado con el notebook. El .py se lee y se revisa mucho mejor en un Pull Request.

Recomendación Para empezar, con nbstripout es suficiente. Cuando el trabajo grupal se intensifique en la Fase 3, añadan el emparejamiento con Jupyter: revisar código en un .py es incomparablemente más claro.

Comprueba Ejecute una celda de su notebook, guárdelo y corra `git diff`. Si ve las salidas de la celda en el diff, nbstripout no está activo.

PARTE VII. Reproducibilidad del entorno

Tiempo de lectura | 8 min · Semana 1, Fase 2

Al terminar esta parte otra persona podrá recrear su entorno de forma idéntica.

En un contexto científico, quien revise su trabajo debe poder reproducirlo. Cada vez que instale o actualice paquetes, guarde la lista exacta:

```
python -m pip freeze > requirements.txt
```

Versione el requirements.txt. Cualquiera podrá reconstruir el entorno con estos tres comandos, en este orden:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Atención — Anaconda Si su Python viene de Anaconda/conda, `pip freeze` puede escribir paquetes con rutas locales (`paquete @ file:///Users/...`) que fallan al instalar en otro equipo con «No such file or directory». Genere la lista con `python -m pip list --format=freeze > requirements.txt`, o escriba a mano solo los paquetes que su proyecto usa.

Atención — equipos mixtos Algunos paquetes tienen versiones distintas según el sistema operativo o la arquitectura (Apple Silicon frente a Intel o Windows). Si

un compañero no logra instalar el `requirements.txt` generado en su Mac, no lo edite a ciegas: dejen en el archivo solo las dependencias que el proyecto realmente importa, sin las transitivas, y anoten en el README la versión de Python usada.

7.1 Reproducibilidad no es solo el `requirements.txt`

Un entorno idéntico no garantiza resultados idénticos. Tres puntos adicionales que la rúbrica revisa bajo Reproducibilidad y Validación técnica:

Rutas relativas, siempre. `pd.read_csv("/Users/camila/Desktop/datos.csv")` funciona en un solo computador del mundo. Use `pd.read_csv("data/raw/datos.csv")` y ejecute el notebook desde la raíz del proyecto.

Semillas aleatorias. Cualquier operación con azar —muestreo, división de datos, inicialización— debe fijar la semilla, o los resultados cambiarán en cada ejecución y nadie podrá verificar los suyos:

```
import numpy as np
RANDOM_SEED = 42
rng = np.random.default_rng(RANDOM_SEED)
```

Ejecución limpia antes de entregar. En JupyterLab: **Kernel → Restart Kernel and Run All Cells**. Si el notebook falla, es que dependía del orden en que usted fue ejecutando celdas, no del orden en que están escritas. Un notebook que solo funciona en un orden distinto al mostrado tiene un problema de reproducibilidad, y es visible en la revisión: los contadores In [7], In [3], In [12] lo delatan.

Nota Existen herramientas modernas (`uv`, `pixi`, `Poetry`) que generan un *lockfile* con versiones exactas y resuelven dependencias mucho más rápido. Para este módulo, `requirements.txt` cumple perfectamente; vale la pena conocerlas para después.

Comprueba ¿Su notebook corre completo tras *Restart Kernel and Run All Cells*, en una carpeta recién clonada, sin editar ninguna ruta?

PARTE VIII. GitHub: cuenta y repositorio del grupo

Tiempo de lectura | 7 min · Semana 1, Fase 1

Al terminar esta parte el grupo tendrá un repositorio remoto con todos los integrantes con acceso de escritura.

8.1 Crear la cuenta

Entre a <https://github.com>, regístrese con **el mismo correo que configuró en Git**, confirme y acceda.

Recomendación Use un nombre de usuario profesional. Sus repositorios pueden formar parte de su portafolio, y este proyecto es de los primeros que podrá mostrar.

8.2 Crear el repositorio remoto

En GitHub: **New repository**, asígnele el nombre acordado (proyecto-grupoX-mcdi500), elija público o privado y créelo. **No marque** la opción de añadir README ni .gitignore si ya los tienen en local: evitarán conflictos en el primer push.

Nota Si lo crean privado, agreguen como colaborador al docente cuando se les indique, o el repositorio no podrá revisarse. Un enlace que el revisor no puede abrir se evalúa como componente no verificable, y en ese caso el criterio no se da por cumplido.

8.3 Agregar a los integrantes del grupo

Una sola persona crea el repositorio. Luego, en GitHub: **Settings → Collaborators → Add people**, e invite a los demás integrantes por su nombre de usuario. Cada uno recibe una invitación por correo que debe aceptar.

Desde ese momento, **todos clonan el mismo repositorio**:

```
git clone https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
cd proyecto-grupoX-mcdi500
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
nbstripout --install
```

Rúbrica Esta es la única forma de que el historial muestre commits de cada integrante. La alternativa habitual —que una persona reciba los archivos por mensajería y los suba— produce un repositorio con un solo autor, que en el criterio de Repositorio GitHub no alcanza el nivel Excelente por más completo que esté el contenido.

Comprueba ¿Cada integrante puede ejecutar `git push` sin error de permisos? Pruébenlo en la primera semana, no la noche de la entrega.

PARTE IX. Conectar Git local con GitHub

Tiempo de lectura | 5 min · Semana 1, Fase 1

Si usted clonó el repositorio (Parte 8.3), el remoto ya está configurado y puede saltar a 9.3. Si empezó en local, siga desde aquí.

9.1 Agregar el remoto

```
git remote add origin https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
```

origin es el nombre convencional del repositorio remoto principal.

9.2 Verificar el remoto

```
git remote -v
```

9.3 Autenticarse con GitHub CLI

Instale la CLI si no la tiene y auténtíquese:

```
brew install gh  
gh auth login
```

Responda: **GitHub.com** → protocolo **HTTPS** → autenticar Git con sus credenciales **Yes** → **Login with a web browser**. Se abrirá el navegador; inicie sesión, autorice y vuelva a la terminal. Verifique con:

```
gh auth status
```

Nota Si usted no tiene Homebrew instalado, puede descargar la CLI desde <https://cli.github.com> o instalar Homebrew primero siguiendo las instrucciones de <https://brew.sh>.

9.4 Subir el proyecto por primera vez

```
git push -u origin main
```

Nota -u deja enlazada la rama local con la remota. Después basta con `git push`.

PARTE X. Flujo de trabajo en equipo

Tiempo de lectura | 6 min · Semana 1, Fase 2

Al terminar esta parte sabrá el ciclo diario y por qué `git pull` va antes que `git push`.

10.1 El ciclo individual

```
git status           # revisar qué cambió  
git add .            # preparar  
git commit -m "Describe el avance" # registrar  
git push             # subir
```

10.2 El ciclo en grupo

Cuando varias personas trabajan sobre el mismo repositorio, hay un paso más, y omitirlo es la causa de la mayoría de los problemas:

```
git pull             # ANTES de empezar a trabajar: traer lo que hicieron los demás  
# ... trabajar ...  
git status  
git add .
```

```
git commit -m "Describe el avance"
git pull          # otra vez: alguien pudo subir algo mientras trabajaba
git push
```

Atención Si al hacer `git push` aparece `rejected ... non-fast-forward`, significa que el remoto tiene commits que usted no tiene. **No fuerce el push con `-force`**: sobrescribiría el trabajo de sus compañeros y sería irrecuperable. Haga `git pull`, resuelva lo que haga falta (Parte XIII) y vuelva a subir.

Recomendación — acuerdo de grupo Repártanse el trabajo por archivo, no por línea. Dos personas editando el mismo notebook a la misma hora garantiza conflictos; dos personas trabajando en `F2/notebooks/limpieza.ipynb` y `F2/src/transformaciones.py` no chocan nunca. Definan esto en la reunión de planificación de cada fase.

Comprueba ¿Sabe qué hace `git pull` que no hace `git fetch`? (Pista: `pull = fetch + merge`.)

PARTE XI. Ramas

Tiempo de lectura | 5 min · Semana 2, Fase 3

Al terminar esta parte podrá trabajar en una funcionalidad sin arriesgar la versión estable.

11.1 ¿Qué es una rama?

Una rama es una línea de trabajo independiente dentro del mismo proyecto. Sirve para desarrollar, corregir o experimentar sin afectar la versión estable. `main` representa esa versión estable: lo que está en `main` debería funcionar siempre.

En el proyecto del curso, la utilidad es directa: si está probando tres formas de imputar valores faltantes, cada una puede vivir en su rama y solo la elegida se integra a `main`.

11.2 Crear y cambiar de rama

```
git switch -c f2-limpieza-datos  # crear y cambiar en un paso
git branch                      # ver ramas (la actual con *)
git switch main                  # volver a la principal
```

Nota Verá también `git checkout` en muchos tutoriales: hace lo mismo, pero `git switch` y `git restore` son la forma moderna y más clara, porque separan «cambiar de rama» de «descartar cambios».

Recomendación Nombre las ramas por lo que hacen y por la fase: `f2-limpieza-datos`, `f3-recursividad`, `f3-clase-dataset`. Evite `rama1`, `prueba`, `camila`.

11.3 Trabajar y guardar en la rama

```
git add .
git commit -m "feat: agrega imputacion de valores faltantes por mediana"
git push -u origin f2-limpieza-datos
```

PARTE XII. Colaboración con Pull Requests

Tiempo de lectura | 6 min · Semana 2, Fase 3

Al terminar esta parte sabrá integrar cambios mediante revisión, no a ciegas.

En GitHub, la forma correcta de integrar y revisar cambios es el **Pull Request (PR)**. Es el mecanismo que usará cuando trabaje en grupo y el que replica lo que ocurre en cualquier equipo de datos profesional.

1. Crear y trabajar en una rama

```
git switch -c f3-medicion-complejidad
# ... editar archivos ...
git add .
git commit -m "feat: mide complejidad temporal de la busqueda recursiva"
```

2. Subir la rama al remoto

```
git push -u origin f3-medicion-complejidad
```

3. En GitHub aparecerá un botón para abrir un Pull Request desde esa rama hacia main. Ábralo, **describa el cambio** y solicite revisión a un compañero.
4. El revisor comenta línea por línea. Cuando esté conforme, se fusiona el PR desde la interfaz de GitHub.
5. Actualice su main local:

```
git switch main
git pull
git branch -d f3-medicion-complejidad
```

Recomendación — cómo revisar un PR de un compañero Revisar no es aprobar por cortesía. Preguntas útiles: ¿el código hace lo que dice el mensaje del commit? ¿las transformaciones están justificadas o simplemente aplicadas? ¿corre si lo ejecuto desde cero? ¿los nombres de variables se entienden sin preguntar? Un comentario concreto en una línea vale más que un «se ve bien».

Rúbrica El PR deja registro de la discusión y de la revisión, no solo del código. Es evidencia directa para el criterio de Documentación del proceso, que pide justificar decisiones técnicas indicando motivo e impacto.

12.1 Fusión local (alternativa simple)

Para trabajo individual o cambios menores:

```
git switch main
git merge f3-medicion-complejidad
git push
git branch -d f3-medicion-complejidad
```

PARTE XIII. Conflictos: qué son y cómo resolverlos

Tiempo de lectura | 6 min · Semana 2, Fase 3

Al terminar esta parte un conflicto dejará de ser una emergencia.

Un conflicto ocurre cuando dos personas modifican **la misma parte del mismo archivo** y Git no puede decidir cuál versión conservar. No es un error ni un castigo: es Git pidiendo que se tome una decisión que solo el equipo puede tomar.

13.1 Cómo se ve

Al hacer `git pull` o `git merge`, Git avisa y marca el archivo así:

```
<<<<<< HEAD
df = df.dropna(subset=["edad"])
=====
df["edad"] = df["edad"].fillna(df["edad"].median())
>>>>>> f2-limpieza-datos
```

Arriba, su versión. Abajo, la que viene de la otra rama.

13.2 Cómo se resuelve

1. Abra el archivo y **converse con su compañero** si el conflicto es de criterio técnico, como en el ejemplo: eliminar filas o imputar la mediana no son equivalentes, y la decisión debe quedar justificada en el informe.
2. Edite el archivo dejando el contenido final correcto y **borre las tres líneas de marcas** (`<<<<<<`, `=====`, `>>>>>>`).
3. Confirme la resolución:

```
git add archivo_en_conflicto.py
git commit -m "fix: resuelve conflicto en estrategia de imputacion"
```

Si desea abortar y volver al estado anterior: `git merge --abort`.

13.3 Conflictos en notebooks

Un conflicto dentro de un `.ipynb` es prácticamente ilegible, porque el archivo es JSON con salidas incrustadas. Tres formas de tratarlo, en orden:

- **Prevenir:** `nbstripout` activo en todos los equipos (Parte VI) reduce drásticamente la frecuencia.
- **Repartir:** que dos personas no editen el mismo notebook simultáneamente.

- **Resolver:** si ya ocurrió, lo más práctico es quedarse con una versión completa —`git checkout --ours archivo.ipynb` o `--theirs`— y reincorporar a mano las celdas de la otra, en vez de intentar editar el JSON.

Comprueba Provoquen un conflicto a propósito en un archivo de prueba y resuélvanlo juntos. Es mucho mejor aprender esto en la Semana 1 que la noche antes de la Sumativa 3.

PARTE XIV. El README técnico

Tiempo de lectura | 6 min · Semanas 1 y 3, Fases 1 y 4

Al terminar esta parte tendrá un README que cumple el criterio de la rúbrica.

El README es lo primero que ve quien abre el repositorio, y en este curso es evidencia evaluada: la rúbrica pide un README **estructurado**, que describa la organización del proyecto, los criterios de versionamiento, las dependencias y los lineamientos de reproducibilidad.

Un README que dice «Proyecto de MCDI500» no cumple. Uno que permite a un tercero ejecutar el proyecto sin preguntar nada, sí.

14.1 Plantilla

Proyecto Grupo X – MCDI500

Breve descripción de la problemática abordada y del objetivo del análisis (2 o 3 frases).

Integrantes

- Nombre Apellido (@usuario-github)
- Nombre Apellido (@usuario-github)

Datos

Fuente del dataset, licencia o condiciones de uso, número de registros y variables. Si el dataset no está versionado: dónde descargarlo y en qué carpeta colocarlo.

Estructura del repositorio

- F1/ Definición del problema y entorno reproducible
- F2/ Obtención, limpieza y transformación de datos
- F3/ Núcleo algorítmico: programación estructurada, recursiva y POO
- F4/ Análisis, visualización y comunicación de resultados

Requisitos y ejecución

Python 3.13

```
python3 -m venv .venv
source .venv/bin/activate # macOS y Linux
```

```
# .venv\Scripts\activate          # Windows
python -m pip install -r requirements.txt
```

Ejecutar los notebooks en orden desde la raíz del proyecto.

Convención de commits

Prefijos usados: docs, data, feat, fix. Ver Parte XV de la guía.

Decisiones técnicas

Registro breve de las decisiones relevantes y su motivo.

Ejemplo: "Se imputa la edad con la mediana en lugar de eliminar registros, porque los NA representan el 12% de la muestra y su eliminación sesgaría la distribución por región."

Recomendación Documenten la activación del entorno **para macOS y para Windows**, como en la plantilla. Es lo que evita que el integrante con el otro sistema quede bloqueado en el peor momento.

Recomendación Actualice el README **al cerrar cada fase**, no al final del curso. La sección de decisiones técnicas escrita en el momento es la que después evita reconstruir de memoria la justificación para el informe.

PARTE XV. Mensajes de commit

Tiempo de lectura | 4 min · Semana 1, Fase 2

Al terminar esta parte sus commits comunicarán algo a quien lea el historial, incluido usted mismo la próxima semana.

15.1 La convención del curso

Use un prefijo que indique el tipo de cambio, seguido de una descripción en presente:

Prefijo	Se usa para	Ejemplo
feat	Funcionalidad o análisis nuevo	feat: agrega funcion de normalizacion de variables numericas
fix	Corrección de un error	fix: corrige indice desalineado tras el merge de tablas
data	Cambios en datos o su procesamiento	data: incorpora dataset de accidentes 2024 en raw
docs	Documentación, README, informe	docs: documenta criterios de limpieza en el README

15.2 Qué distingue un buen mensaje

Evite	Prefiera
cambios	feat: agrega deteccion de duplicados por RUT

Evite	Prefiera
avance	data: elimina 47 registros duplicados del dataset base
arreglado	fix: corrige lectura de fechas con formato dd-mm-yyyy
subo notebook	docs: documenta decisiones de imputacion en notebook F2

La prueba es simple: **si alguien lee solo su historial, ¿entiende cómo evolucionó el proyecto?** Ese es literalmente el ejercicio que hace quien evalúa.

Recomendación Commits pequeños y frecuentes, no uno enorme al final del día. Un commit debería corresponder a un cambio con sentido propio: «agrega la limpieza de nulos» es un commit; «trabajo del martes» son ocho commits mezclados.

Atención — evite tildes y caracteres especiales en los mensajes de commit. Aunque macOS los maneja sin problemas, el equipo puede incluir computadores con Windows donde, según la configuración de la terminal, aparecen corruptos en el historial compartido.

PARTE XVI. Errores comunes y su significado

Tiempo de lectura | consulta puntual

command not found: python

En macOS el comando python a secas no existe mientras no haya un entorno virtual activo. Use python3, o active el entorno con `source .venv/bin/activate` y a partir de ahí python funciona.

error: externally-managed-environment

Usted intentó instalar paquetes en el Python del sistema, que macOS protege. La solución correcta **no** es forzar la instalación, sino activar el entorno virtual del proyecto antes de instalar:

```
source .venv/bin/activate
python -m pip install nombre_paquete
```

fatal: unable to auto-detect email address

Git no tiene configurado nombre o correo.

```
git config --global user.name "Nombre"
git config --global user.email "correo@ejemplo.com"
```

src refspec main does not match any

Todavía no existe ningún commit en main.

```
git add .
git commit -m "Primer commit"
git push -u origin main
```

remote origin already exists

Ya hay un remoto llamado origin.

```
git remote set-url origin NUEVA_URL
```

Updates were rejected because the remote contains work that you do not have locally

El remoto tiene commits que usted no tiene. **No use --force.**

```
git pull
# resolver conflictos si aparecen
git push
```

xcrun: error: invalid active developer path

Aparece tras actualizar macOS: las Command Line Tools quedaron desvinculadas. Reinstálelas con `xcode-select --install`.

zsh: command not found: brew

Homebrew no está instalado o no está en el PATH. Instálelo siguiendo las instrucciones de <https://brew.sh> y reinicie la Terminal.

ModuleNotFoundError con un paquete que sí instaló

Casi siempre el entorno virtual no está activo, o el notebook está usando el kernel «Python 3» genérico en lugar del kernel del proyecto. Verifique que vea (`.venv`) en la terminal y revise el kernel en la esquina superior derecha de JupyterLab.

this exceeds GitHub's file size limit of 100.00 MB

Intentó subir un archivo demasiado grande, normalmente un dataset. Agréguelo al `.gitignore` y documente en el README dónde obtenerlo. Si ya lo registró en un commit, consulte en el Foro de Consultas antes de intentar reescribir el historial por su cuenta.

Subió por error .venv/, .DS_Store o un archivo con credenciales

Agréguelo al `.gitignore` y quítelo del rastreo sin borrarlo del disco:

```
git rm -r --cached .venv
git rm --cached .DS_Store
git commit -m "fix: deja de versionar archivos que no corresponden"
```

Si eran credenciales reales, además **cámbielas**: siguen visibles en el historial.

Nota Si el error que aparece no está en esta lista, publíquelo en el **Foro de Consultas** indicando el comando ejecutado, lo que esperaba y el mensaje exacto. Es muy probable que otro grupo esté enfrentando lo mismo.

PARTE XVII. Integridad académica en el repositorio

Tiempo de lectura | 3 min

El historial de Git es un registro de autoría. Eso tiene dos implicancias directas en este módulo.

El trabajo debe ser propio. Las guías de apoyo del curso entregan esqueletos con marcas TODO para orientar, no soluciones para copiar; el plagio se sanciona con nota mínima. Un repositorio cuyo primer commit contiene el proyecto completo y terminado es, como mínimo, una señal de que el proceso no ocurrió como se declara.

El uso de asistentes de IA debe ser transparente. Si utilizó un asistente para generar o depurar código, declárelo según los lineamientos de uso ético de la IA publicados en el curso y —esto es lo importante— asegúrese de **entender y poder explicar cada línea que entrega**. La evaluación distingue entre obtener un resultado correcto y comprender por qué es correcto: un código sofisticado acompañado de una explicación que evidencia desconocimiento se observa como inconsistencia académica.

La contribución individual es visible. El historial muestra quién hizo qué y cuándo. Esto protege a quien trabaja, y por eso el criterio de la rúbrica lo exige.

PARTE XVIII. Checklist antes de cada entrega

Tiempo de lectura | 4 min · antes de cada evaluación

Revise esta lista **con el repositorio abierto**, no de memoria.

Repositorio - ☐ La estructura de carpetas corresponde a las fases entregadas (F1, F2, ...). - ☐ .gitignore presente y funcionando: .venv/ y .DS_Store no aparecen en GitHub. - ☐ README actualizado, con estructura, dependencias, instrucciones de ejecución para macOS y Windows, y decisiones técnicas de la fase. - ☐ requirements.txt regenerado después de la última instalación de paquetes. - ☐ Todo lo local está subido: git status responde *nothing to commit, working tree clean*.

Historial - ☐ Hay commits de **cada integrante** del grupo. - ☐ Los mensajes describen el cambio y usan los prefijos de la convención. - ☐ Los commits están repartidos en el tiempo, no todos en las últimas horas.

Notebooks - ☐ Corren completos tras *Restart Kernel and Run All Cells*. - ☐ Usan rutas relativas. - ☐ Fijan semilla aleatoria donde hay azar. - ☐ Las celdas narrativas explican **por qué** se hizo cada paso, no solo qué se hizo. - ☐ Se adjunta la versión ejecutada (PDF o HTML) si el repositorio guarda los notebooks limpios.

Acceso - [] El enlace del repositorio está en el informe. - [] Si es privado, el docente tiene acceso. **Verifíquelo abriendo el enlace desde una sesión distinta.**

Atención Un componente que el revisor no puede verificar no se asume cumplido. Un repositorio privado sin acceso equivale, para efectos de la evaluación, a un repositorio ausente.

PARTE XIX. Secuencia consolidada desde cero

Tiempo de lectura | consulta puntual

19.1 Del inicio a la publicación

```
# Configuración (una sola vez por equipo)
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
git config --global init.defaultBranch main

# Crear proyecto y entorno
mkdir proyecto-grupoX-mcdi500 && cd proyecto-grupoX-mcdi500
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install jupyterlab numpy pandas matplotlib seaborn \
    scikit-learn jupyter nbstripout

# Inicializar Git y limpiar notebooks
git init
nbstripout --install

# Crear archivos (incluido .gitignore CON contenido)
touch README.md
# ... escribir el .gitignore y la estructura de carpetas ...

# Guardar dependencias y primer commit
python -m pip freeze > requirements.txt
git add .
git commit -m "docs: crea estructura inicial del proyecto"

# Conectar con GitHub y subir
git remote add origin https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
git push -u origin main
```

19.2 Incorporarse a un repositorio existente (resto del grupo)

```
git clone https://github.com/USUARIO/proyecto-grupoX-mcdi500.git
cd proyecto-grupoX-mcdi500
python3 -m venv .venv
source .venv/bin/activate
```

```
python -m pip install -r requirements.txt
nbstripout --install
```

19.3 Desarrollo con ramas y Pull Request

```
git pull
git switch -c f2-limpieza-datos
# ... trabajar ...
git add .
git commit -m "feat: agrega pipeline de limpieza de valores faltantes"
git push -u origin f2-limpieza-datos
# Abrir el Pull Request en GitHub, revisar y fusionar
git switch main
git pull
git branch -d f2-limpieza-datos
```

Glosario

Término	Significado
Repositorio	Carpeta del proyecto con su historial de Git.
Commit	Registro guardado de un conjunto de cambios, con autor, fecha y mensaje.
Staging	Zona intermedia donde se preparan los cambios del próximo commit.
Rama (branch)	Línea de trabajo independiente dentro del mismo repositorio.
main	Rama principal; la versión estable del proyecto.
origin	Nombre convencional del repositorio remoto en GitHub.
Clonar	Descargar una copia completa de un repositorio remoto, con su historial.
Push / Pull	Subir los commits propios al remoto / traer los commits del remoto.
Pull Request (PR)	Propuesta de integrar una rama en otra, abierta a revisión y discusión.
Merge	Fusión de dos ramas.
Conflicto	Situación en que dos versiones modifican la misma parte de un archivo y Git pide decidir.
Kernel	Intérprete de Python que ejecuta un notebook.
Entorno virtual	Instalación de Python aislada, propia de un proyecto.
Homebrew	Gestor de paquetes de macOS; instala herramientas como git o gh desde la Terminal.
Command Line Tools	Conjunto de herramientas de desarrollo de Apple, necesarias para compilar librerías científicas.

Autoevaluación

Tiempo de lectura | 5 min

Si usted puede responder estas preguntas sin consultar la guía, domina lo que el módulo espera. Si alguna se le resiste, la columna de la derecha indica dónde volver.

Pregunta	Volver a
1. ¿Qué diferencia hay entre <code>git add</code> y <code>git commit</code> ?	Parte IV
2. ¿Por qué en macOS se usa <code>python3</code> antes de activar el entorno y <code>python</code> después?	Partes II y III
3. ¿Por qué <code>git pull</code> debe ir antes de <code>git push</code> cuando trabaja en grupo?	Parte X
4. ¿Qué problema específico de los notebooks resuelve <code>nbstripout</code> ?	Parte VI
5. ¿Por qué no basta con subir el <code>.ipynb</code> para que otra persona reproduzca su análisis?	Parte VII
6. Su compañero hizo push y ahora su push es rechazado. ¿Qué hace, y qué no debe hacer nunca?	Partes X y XVI
7. ¿Qué evidencia del trabajo en equipo queda registrada en el historial que no queda en el informe?	Partes VIII y XVII

Estas preguntas son también un buen punto de partida para sus intervenciones en el **Foro de reflexión** de cada fase, donde se espera que fundamente decisiones técnicas con ejemplos de su propia práctica.

Conclusión

Git y GitHub no son solo herramientas técnicas: son parte del modo profesional de desarrollar y de investigar. Aportan orden, trazabilidad y trabajo colaborativo real. En ciencia de datos suman, además, dos cosas que distinguen un trabajo serio de uno improvisado: **reproducibilidad** —cualquiera puede recrear su entorno y sus resultados— e **higiene de notebooks** —un historial limpio, sin salidas pesadas ni datos filtrados.

En este módulo, el repositorio no es un anexo del proyecto: es donde queda registrado que el proyecto ocurrió de la forma en que ustedes lo relatan.

La idea central Ciclo diario: **traer** (`git pull`) → **trabajar** → **revisar** (`git status`) → **preparar** (`git add`) → **guardar** (`git commit`) → **subir** (`git push`).
Cuando el proyecto crece: **crear rama** → trabajar → **commitear** → **subir rama** → **abrir Pull Request** → **revisar** → **fusionar**.